

Turn noise into signal

Senior Full-Stack AI Engineer · Incubator Team

Thanks for making the time. This is a small, self-contained build that mirrors what our Incubator team does every day: take messy real-world data and turn it into a trustworthy, structured product. We care far more about your **judgment and how you work** than about polish or 100% completeness.

What you're given

Attached is `reviews.json` — about 300 customer reviews for five products, pulled from five different channels. It's deliberately messy: typos, several languages, spam, near-duplicates cross-posted between channels, and everything from five-word blurbs to full essays. Each row looks like this:

`reviews.json` — one row

```
{
  "id":      "r_6c4b9c9a73",
  "product": "AeroBuds (earbuds)",
  "source":  "trustpilot",      // one of five channels — see below
  "rating":  1,                 // 1-5, or null (some channels have no star rating)
  "language": "en",             // en, plus some es / de / pt / fr / zh
  "text":    "Regret paying for this. Missing half the order, and it won't hold a charge."
}
```

`channels:` app_store · google_play · trustpilot · post_purchase_email · support_chat

Your task

Build a small app that:

- 1 **Extract** — for each review, use an LLM to pull out its overall **sentiment**, the specific **issues / topics** raised, and a **severity**.
- 2 **Handle the mess** — filter out spam / junk, collapse the near-duplicate cross-posts into one, and still classify the non-English reviews rather than dropping them.
- 3 **Aggregate** — roll it up into a “**top issues**” view: counts and a couple of representative example quotes per issue, broken down per product.
- 4 **Show it** — a small but real **UI** where a non-technical person can see the top issues, **filter** (by product / severity), and click through to the underlying reviews. (A CLI is acceptable only if the output is genuinely readable — but a small UI is what we're hoping to see.)

- 5 **Prove it works** — hand-label about **20 reviews** yourself, include a runnable script or section that reports how well your system **agrees** with your labels, and write a short paragraph on **where it fails and why**.

What to send back

DELIVERABLES

- A **Git repo** (link or zip) that runs from a clean clone, with a **README**: setup, how to run, and how to run the eval.
- A one-page **WRITEUP.md**: your architecture, the 2–3 biggest decisions & trade-offs, what you'd do with another week, and what you deliberately cut.
- An **AI_USAGE.md**: which AI tools you used, what you delegated vs wrote yourself, 2–3 example prompts, and one place the AI got it wrong and how you caught it.

HOW WE'D LIKE YOU TO WORK

- **Time-box: ~4–6 focused hours.** Please don't gold-plate — skipping something and saying so in the writeup is a signal, not a mark against you.
- **Any stack, any LLM provider.** We'll give you an API key on request — or use your own, a local model, or a clean mock.
- **Use your AI tools.** We build with Cursor / Claude / Copilot daily and expect you to. The scope is broad on purpose — lean on the tools to move fast, and stay in control of what ships.
- **Please don't commit secrets.** Auth, deployment, and multi-tenant are out of scope.

What happens next

A relaxed **30-minute call**: you walk us through the repo, we ask about a couple of your decisions, and we'll ask you to make one small change live using your own tools. Nothing to prepare — just bring the project.

When you're done, reply with your repo link (or a zip). Any questions along the way, just ask — knowing when to ask a sharp question is part of the job. Have fun with it.